迷你搜索引擎项目报告

摘要

本项目用 C++17 实现了一个可对《莎士比亚全集》进行查询的迷你搜索引擎，完成了题目要求的词频统计与停用词识别、带词干化的倒排索引、单词/短语（及布尔）查询，以及查询阈值实验，并对”50 万文件、4 亿不同词” 的附加题给出了实测与可复现的外推。

语料从 MIT Shakespeare 站点镜像的 1 211 个 HTML 文件中按站点首页清单抽取为 195 个文档（37 部剧、154 首十四行诗、4 首长诗），共 152 632 行、930 911 个词，默认配置切出 866 626 个正文词元；另以 3 篇 IETF RFC 作为第二语料检验引擎的语料无关性。

在词频统计部分，我们用 tf/df 双指标与一条可扫描的候选规则划定停用词边界，最终得到 39 个词；它们只占词表的一小部分，却覆盖 35.74% 的正文词元。在此基础上引入文档分布熵与长度归一化速率熵，把人工审核升级为” 宽、重、平” 三特征规则：纯统计推导同样得到 39 个词，与人工表重叠 35 个；在 RFC 语料上推导出完全不同的一张停用词表（58 个词），而 RFC 2119 关键词 `must/may/should` 因分布不够” 平” 被正确排除。

在索引部分，Porter 词干化把 25 705 个词形归并为 14 947 个词干；正式索引含 14 937 个词项、117 704 条倒排记录和 556 866 个位置。第一版索引格式（v1，未压缩）为 3 685 655 字节；第二版索引格式（v2，差分加变长编码）压缩到 1 725 694 字节（46.82%）。伸展树、哈希表与 `std::map` 三种词典实现分别构建出逐字节相同的索引；磁盘 B+ 树附属词典文件（237 568 字节）把冷查询 I/O 降到全量加载的 0.225%；SPIMI 式块式构建在 17 个块内完成，产物与内存构建逐字节一致。

查询程序支持单词、短语与布尔查询，并提供 `max-df`、`min-df`、`min-match`、`top-x` 四个阈值。在 15 条查询、201 条测量构成的阈值矩阵上，我们观察到阈值改变的是查询语义而非单纯精度：丢弃查询词会让全部命中退化为单词查询，结果反而从 30 篇增加到 90 篇；短语查询不受查询词裁剪影响，而停用词查询在过滤索引上恒为空，这些都是可解释的设计行为。

正确性方面，18 项 CTest 全部通过（8 个 Catch2 可执行文件、46 个用例、85 155 条断言），并以独立 Python 实现完成了词频 CSV 的逐字节对拍、正式/原始/压缩三种索引的全词表文档频率、词频、文档编号与位置对拍、303 条短语对拍与磁盘词典双路径校验，讲义四文档黄金样例逐项吻合。附加题方面，50 万文档、2 000 万词元的合成语料用块式构建完成（峰值内存 1.01 GiB，内存构建为 2.00 GiB，产物逐字节相同）；B+ 树词典从 10 万键实测到 5 000 万键，体积恒定 15.100 字节/键，5 000 万键时树高已达 4；按此外推 4 亿词约需 6.04 GB 磁盘、树高 4，单次查询只读 4 个页面（约 16 KB）。

关键词： 倒排索引；词频统计；停用词；速率熵；Porter 词干化；伸展树；B+ 树；查询阈值；可复现性

目录

摘要 **1**

1 设计流程 **4**

1.1 语料收集 4

1.1.1 语料来源 4

1.1.2 预处理与切词 4

1.1.3 特殊情况处理 5

1.2 词频统计 5

1.2.1 设计原理 5

1.2.2 算法设计 5

1.3 停用词选择 5

1.4 词干化 6

1.4.1 网上方案选择与分析 6

1.4.2 算法设计 6

1.5 倒排索引 6

1.6 词典结构 7

1.7 磁盘 B+ 树和块式构建 7

1.8 查询设计与阈值设计 8

2 测试与结果分析 **9**

2.1 词频统计与停用词选择 9

2.1.1 词频统计结果 9

2.1.2 停用词选择结果 10

2.1.3 自动推导结果 10

2.2 倒排索引与查询结果 11

2.2.1 倒排索引构建结果 11

2.2.2 查询结果 12

2.3 阈值实验与复杂度分析 14

2.3.1 阈值实验结果 14

2.3.2 复杂度分析 16

3 Bonus 的结果与分析 **16**

3.1 规模边界与实测范围 16

3.2 磁盘 B+ 树词典 17

3.3 块式构建与 500,000 文档 17
3.4 第二版索引格式的压缩结果 18
3.5 Bonus 结论 18

4 项目评价 **19**
4.1 推广：第二语料 RFC 19
4.2 局限与改进 19

5 参考文献 **21**

1 设计流程

题目要求完成词频统计、带词干化的倒排索引、单词与短语查询，以及查询阈值实验。这些工作不能彼此分离：停用词改变位置编号，词干化改变词项，查询又必须复用建索引时的规范化规则。项目因此按“语料收集、词频统计、停用词选择、词干化、倒排索引、词典结构、外部构建、查询”的顺序展开，每一阶段都留下可重跑的命令和输出。

系统的总数据流如图 1：预处理阶段使用 Python 把 HTML 转成文本层，再交由 C++ 进行处理；切词后的词元同时进入统计与索引两条路径；查询侧按索引头记录的处理标志重放停用词与词干化步骤。这样可以把网页解析错误、统计口径错误和索引错误分开定位。

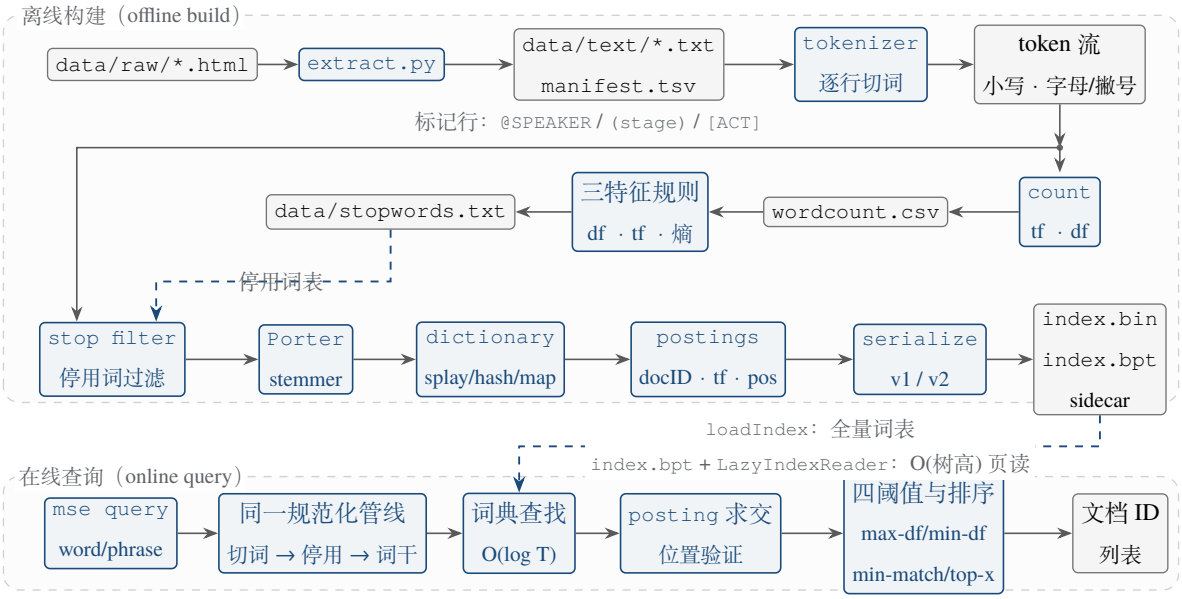


图 1: 系统总体数据流：离线侧把 HTML 语料抽取为文本层，经切词、停用词过滤与 Porter 词干化后写入倒排索引，词频统计与停用词推导为旁路；在线侧复用同一规范化管线，经词典查找、posting 求交与位置验证、四个阈值后返回文档。虚线为 `--dict-file` 生成的 B+ 树 sidecar，查询只读 O(树高) 页。

1.1 语料收集

1.1.1 语料来源

主语料取自 MIT Shakespeare 站点 [1] 及其官方 GitHub 镜像。仓库中同时存在现行小写作品目录、旧版大写分类目录和 Moby 纯文本版本，我们以 MIT Shakespeare 站点首页的目录作为作品清单。

1.1.2 预处理与切词

`tools/extract.py` 把每个 HTML 页面转换成“一行一个语义单元”的文本文件，同时生成 `manifest.tsv` 作为整个语料的文档目录和映射表。

考虑到语料为英文，因此切词器把“连续 ASCII 字母并允许内部撇号”的极大串定义为词元。`don't` 与 `o'er` 保持为一个词，行首撇号被丢弃（例如`'Tis`→`tis`），行尾撇号作为分隔（例如 `friends'`→`friends`）；连字符、数字和普通标点均为分隔符（例如 `to-day`→`to,day`，`1660s`→`s`）。所有词元均进行小写化。

1.1.3 特殊情况处理

处理过程中经过检验发现，莎士比亚语料中存在一些特殊情况：

舞台提示格式不统一 例如 Moby 版把内嵌舞台提示写成方括号。

HTML 文件本身标签残缺 例如 `Poetry/elegy.html` 中 `</TITLE\n </HEAD>` 由于缺少闭合尖括号而导致读取失败。

我们都进行了额外的判定。

1.2 词频统计

1.2.1 设计原理

统计程序回答两个基础量：词频 tf（词项在语料中的总出现次数）与文档频率 df（包含该词项的文档数）。对词项 t：

$$\mathrm{tf}(t) = \sum_{d} \mathrm{tf}(t,d), \qquad \mathrm{df}(t) = |\{d : \mathrm{tf}(t,d) > 0\}|. \tag{1}$$

同时计算 $df_ratio(t) = \mathrm{df}(t)/195$ 与 $tf_share(t) = \mathrm{tf}(t)/866\,626$。前一个指标描述“覆盖面”，后一个描述“词元质量”；只看其中一个，很容易把短诗稀释的对话词误判为稀有词。

统计程序复用正式规范化管线：切词后可选停用词过滤，再可选 Porter 词干化。默认、仅停用词、仅词干化和正式配置分别对应四种统计数据，后续索引与查询均使用同一组处理顺序。

1.2.2 算法设计

程序按语料清单顺序逐文档扫描。每个词项除累计词频外，还保存最近一次出现的文档编号；只有当前文档编号与 `lastDoc` 不同时，文档频率才加一。这样同一篇作品内的重复出现不会重复计算文档频率：

```
for each document d:
    for each token t in tokenize(d):
        row = table[t]
        row.tf += 1
        if row.lastDoc != d.id:
            row.df += 1
            row.lastDoc = d.id
```

1.3 停用词选择

统计学手段进行选择 一般的候选规则只要求“广”和“重”：

$$df_ratio(t) \geq 0.75 \quad \wedge \quad tf_share(t) \geq 0.001. \tag{2}$$

它只得到 9 个词。考虑到 154 首的短诗占文档总数 79%，很多对话功能词不会出现在每一首诗里，df 比值被稀释。例如 `a` 只有 70.77%，刚好落在 0.75 线下；如果将阈值放宽到 0.40 后则得到 48 个候选，但 `love`、`time` 等有内容的词也进入候选带。

为了把“均匀分布在很多文档中”与“只是总次数高”分开，我们计算文档长度归一化速率熵。设 L_d 为文档 d 的保留词元数，

$$r_{t,d} = \frac{\mathrm{tf}(t,d)}{L_d}, \qquad q_{t,d} = \frac{r_{t,d}}{\sum_{d'} r_{t,d'}}, \qquad \widehat{H}(t) = \frac{-\sum_d q_{t,d} \log_2 q_{t,d}}{\log_2 \mathrm{df}(t)}. \tag{3}$$

长度归一化后，功能词稳定在 0.95 以上，`love`、`king`、`heart` 等内容词明显更低，df=1 的专名定义为 0。

最终规则同时要求“宽、重、平”：

$$df_ratio \geq r \quad \wedge \quad tf_share \geq s \quad \wedge \quad \widehat{H} \geq h. \tag{4}$$

默认 $(r, s, h) = (0.75, 0.001, 0.93)$ 只产生 9 词；实验参数 $(0.40, 0.001, 0.93)$ 产生 39 词，与人工表重叠 35 个。自动表多收 `hath/o/thee/thou`，少收 `more/or/than/their`，差异可以由速率熵逐词解释。

1.4 词干化

1.4.1 网上方案选择与分析

我们参考并比较了四份词干化方案：Porter 1980 论文、作者官网的 canonical C 实现、NLTK 的 `PorterStemmer` 三种兼容模式，以及 Snowball 的 Porter2 算法 [2, 3, 4, 5]。

最终选择采用 Porter 1980 论文和 canonical C 的三处 `--DEPARTURE--`”。三处差异是：两字母及以下的词不改，`bli` 改为 `ble`，`logi` 改为 `log`。

1.4.2 算法设计

进入 Porter 规则前先处理收缩词：按最长匹配剥掉 `n't/'s/'d/'ll/'re/'ve/'t/'m`，再删除剩余撇号。因此 `o'er` 变为 `oer`，`marry'd` 变为 `marryd`。不规则拼写保留其规则结果，例如 `can't`→`ca`、`won't`→`wo`。

Porter 主流程依次执行 step 1a、1b、1c、2、3、4、5a、5b。核心条件是 measure m（V-C 序列数）以及 `*v*`、`*d`、`*o` 谓词。实现遵循“首个匹配后缀说了算”：某个后缀匹配后，只检查该条规则的条件；条件失败就停止，不再退回尝试更短后缀。

1.5 倒排索引

每个词项对应一条按文档编号升序的倒排记录表。记录项保存 (文档编号, 位置起点, 词频)，其中 `posStart` 是倒排记录项在独立位置数组中的起点，`tf` 是本文档内的出现次数。位置采用停用词过滤后的词元偏移，因此短语查询可以在不回到原文的情况下验证相邻关系。

索引加载时检查四类结构不变量：文档编号稠密为 1 到 N，同一倒排记录表内严格递增；每条倒排记录项的词频等于其位置个数；位置总数等于词元总数；词项表严格按字典序排列。构建过程按文档流增量追加，最后将词典冻结为有序词项表：

```
for each document D:
    pos = 0
    for token t in tokenize(D):
        if stop_words.contains(t): continue
        term = stem(t) if stem_flag else t
```

```
        dict.upsert(term).append(D.id, pos)
        pos += 1
    index.docs.push({D.id, D.title, pos})
index.terms = dict.sorted_entries()
```

正式配置得到 14,937 个词项、117,704 条倒排记录、556,866 个位置和 3,685,655 字节的第一版索引格式，文件布局如图 2。



图 2: 索引文件布局：`index.bin` 依次存放 header、doc 表、term 表与 postings 段，term 表记录用 `offset`/`bytes` 指向该词的 postings 块；块内每条 posting 先存 docID 与 tf，再紧跟 tf 个升序位置，因此短语查询只需顺序比较相邻位置。

持久化格式也属于倒排索引设计的一部分。第一版索引格式（代码中记为 v1，未压缩）顺序写出文件头、文档表、词项表和倒排记录块，写入前预计算所有偏移；读取端按同一规格解析并校验不变量。这样同一索引对象总能写出同一字节序列。

第二版索引格式（代码中记为 v2，差分压缩）只替换倒排记录区：文档编号存差分，位置存差分，词频与差分值使用 1–4 字节的字节对齐变长码；文件头、文档表和词项表保持不变。正式索引由 3,685,655 字节降到 1,725,694 字节，为第一版格式的 46.82%，查询输出不变。

1.6 词典结构

构建期词典接口只有三个操作：`upsert`、`find`、`sortedEntries`。主实现是自实现的伸展树。

主结构选伸展树的原因是本题操作序列比较特殊。索引只插不删，红黑树最复杂的删除情形不存在；词流呈 Zipf 分布，访问后把节点旋到根，可以让高频词长期靠近树顶；构建结束时中序遍历直接得到字典序词项表，正好用于冻结有序词典。

1.7 磁盘 B+ 树和块式构建

一万五千词以内，词项表与倒排记录可以整体放入内存；但是在四亿词的规模下，这是很难实现的。项目为此实现两条扩展路径：查询侧使用磁盘 B+ 树附属词典文件，构建侧使用 SPIMI 式块式外部构建。

磁盘附属词典文件把每个词项映射到倒排记录偏移和块长。B+ 树节点按 4 KiB 页预算序列化，叶子用 `nextLeaf` 串成有序链。一次点查只读取树根到叶子的 $O(h)$ 个页，再 seek 到主索引中的倒排记录块，不加载完整词项表。内部路由使用左闭区间 $[\text{key}_{i-1}, \text{key}_i)$，因为分隔键是右子树最小键的真实副本。变长键下按字节中点分裂，并用 `pageSize/2 - maxEntry` 作为可证的填充下界。

块式构建把内存中的活词典限制为预算 B。程序按文档流扫描，达到 `--block-terms` 后排序写临时块，再对所有临时块做按词归并。块边界只落在文档之间，所以后一块中同一词的文档编号一定更大，归并时可以直接拼接倒排记录。正式语料预算 4,096 时产生 17 个临时块，峰值活词 5,129，产物与全内存构建逐字节一致。

1.8 查询设计与阈值设计

查询文本先执行与建索引相同的分词器、停用词和词干化步骤。索引头标志决定是否过滤停用词、是否调用词干化器；被过滤、未登录或被阈值丢弃的词项都会写入说明信息。

单词查询要求归一后恰好剩一个词项；多词查询使用 `--words`，短语查询使用 `--phrase`。短语先选文档频率最小的词作为候选文档，再对候选文档验证后续词是否位于 `start+1`, `start+2`, ...。位置数组按升序保存，所以每一步使用二分查找。

布尔查询默认要求全部可检索词命中。评分向量为 `(matched, tfSum, docID)`，依次按降序、降序、升序排序，保证同一查询结果确定。`--min-match k` 表示至少命中 k 个词，超过可用词数时钳制到词项数。

四个阈值全部在查询期生效，索引只构建一次：

表 1: 查询阈值的作用位置与默认值

参数	默认值	作用位置	含义
`--max-df`	1.0	查询词筛选	丢弃 df 比值超过阈值的词
`--min-df`	1	查询词筛选	丢弃 df 低于阈值的词
`--min-match`	0	布尔条件	至少命中的词数；0 等价于全部命中
`--top-x`	0	结果截断	排序后只保留前 x 篇

短语查询有意绕过 `--max-df` 和 `--min-df`：短语必须整段匹配，删除任何一个词都会改变语义。停用词查询同样始终为空，因为停用词从一开始就没有进入索引；程序用说明信息区分“停用词”和“词典中不存在”两种情况。

2 测试与结果分析

2.1 词频统计与停用词选择

2.1.1 词频统计结果

默认配置只统计正文行。195 个文档共切分出 866 626 个词元，其中不同词项 25 705 个，仅出现一次的词项 9 753 个。核心统计量见表 2。

表 2: 默认配置下的词频统计结果

指标	数值
文档数	195
正文词元数	866 626
不同词项数	25 705
仅出现一次的词项数	9 753

默认配置下词频最高的 10 个词项见表 3。

表 3: 默认配置下词频最高的 10 个词项（含文档频率与两种占比）

排名	词项	词频	文档频率	文档频率比值	词频占比
1	the	28 436	177	0.9077	0.032812
2	and	25 553	189	0.9692	0.029486
3	i	21 131	154	0.7897	0.024383
4	to	20 302	187	0.9590	0.023426
5	of	16 976	175	0.8974	0.019589
6	a	14 527	138	0.7077	0.016763
7	you	13 998	75	0.3846	0.016152
8	my	12 934	159	0.8154	0.014925
9	that	11 513	172	0.8821	0.013285
10	in	11 479	181	0.9282	0.013246

注：文档频率比值为文档频率/195，词频占比为词频/866 626；数值取自 `wordcount-default.csv` 的权威副本。

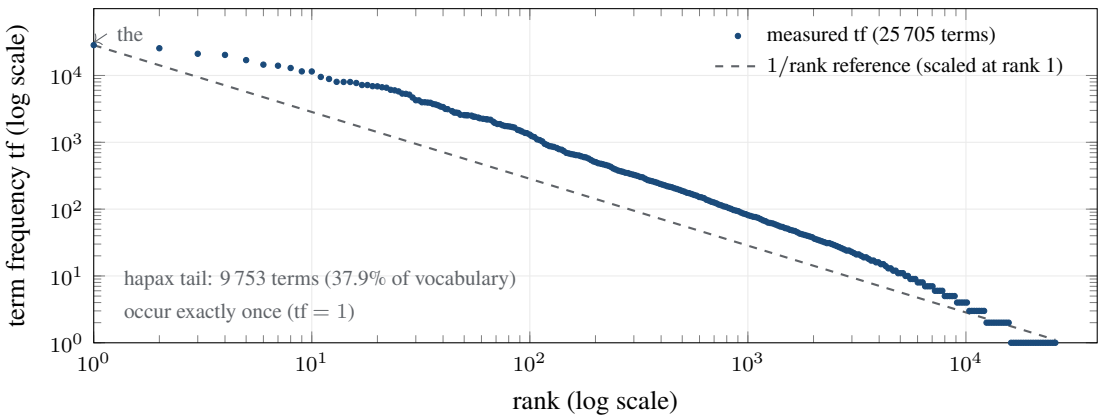


图 3: 莎士比亚语料的词频—排名分布（双对数）：实测点近似一条直线，符合 Zipf 定律；25 705 个词中有 9 753 个只出现一次（占词表 37.9%），构成需要倒排索引兜住的长尾。

前十个词几乎全是功能词，合计占 20.41% 的词元。图 3 的双对数曲线进一步显示，高频词集中在曲线左端，绝大多数词项位于右端的低频长尾。仅出现一次的词占到全部词项的 37.94%，却只覆盖 1.125% 的词元，说明语料具有明显的 “短词表头部、长低频尾部” 特征。由此不能仅凭词频高低判断停用词，下一步还需要比较文档覆盖范围和文档间分布。

2.1.2 停用词选择结果

表 4 是候选规则对文档频率阈值的敏感性扫描：固定词频占比 ≥ 0.001，把文档频率比值阈值从 0.90 降到 0.40。

表 4: 停用词候选带对文档频率比值阈值的敏感性（固定词频占比 ≥ 0.001）

文档频率比值阈值 T	候选数	相对上一档新增	新增词
0.90	4	—	`the and to in`
0.75	9	+5	`i of my that but`
0.70	14	+5	`a is not for with`
0.65	16	+2	`so love`
0.60	21	+5	`me be thou thy thee`
0.50	31	+10	`it this as by all do on from which when`
0.40	48	+17	`his have what no if are o or more then their hath than yet mine time doth`

注：规则为文档频率比值 $\geq T$ 且词频占比 ≥ 0.001；静态 39 词表就是在 0.40 档 48 词候选带上人工取舍的结果。

0.75 档只有 9 个纯功能词；0.65 档内容词 `love` 进入候选（文档频率比值 0.651）；0.40 档就混入 `time`、`thou/thee` 等不该删除的词。纯统计规则能可靠框出” 最像停用词” 的一小段，却无法自己决定线的位置——静态表是在 48 词候选带上人工保留结构性功能词的结果，下一节用速率熵自动完成同一步。

2.1.3 自动推导结果

两特征候选带把最后一步留给人工审核；加上” 平”（速率熵）后，规则能否自动得到与人工表几乎相同的停用词？表 5 是四组阈值与 39 词静态表的双向差集。

表 5: 三特征停用词推导的阈值扫描：与静态 39 词表的差异

(文档频率比值, 速率熵)	推导词数	重叠	静态表独有	推导表独有
(0.75, 0.93)	9	9	30	—
(0.50, 0.95)	23	23	16	—
(0.40, 0.93)	39	35	`more or than their`	`hath o thee thou`
(0.30, 0.93)	50	35	`more or than their`	`at hath he him how let now o thee there thou was well would you`

注：词频占比固定为 ≥ 0.001，文档频率比值取第一列；(0.75,0.93) 档 9 词与两特征候选带一致，(0.40,0.93) 档复现静态表所在的 39 词带。

(0.40, 0.93) 档推导出 39 词、与静态表重叠 35 个；差异恰好是 4 对可解释的词：自动推导多收 `hath o thee thou` 四个古体功能词，静态表多收 `more or than their` 四个兼作比较、并列用的词。前者在原文本中同样” 宽而平”，后者是人工按语义判断的取舍。也就是说，三特征规则把” 是不是停用词” 从人工经验变成可复算的判定，结果与人工表只差 4 个词且差异可逐

词解释；换一档阈值只改变严格度（9/23/39/50 词），不改变规则结构。四个输出文件都带出处头且不含时间戳，换参数可以直接重跑比较。

图 4 把功能词、内容词与专名的速率熵画成条形图并标出 0.93 阈值线。

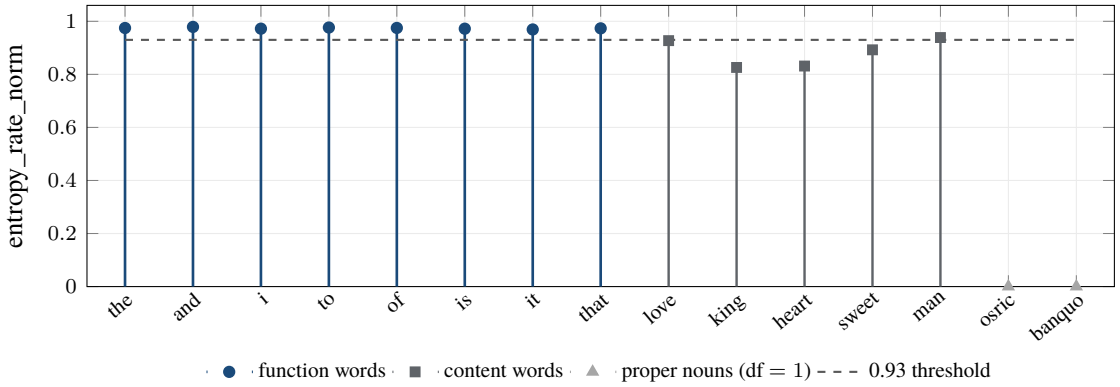


图 4: 速率熵判别：功能词全部贴近 1（均在 0.96 以上），专名（df = 1）按定义为 0，内容词落在 0.83–0.94 的低区间；0.93 一条横线即可把功能词与其余词分开，个别越线的内容词再被 df 比例门（≥ 0.4）挡住，说明该特征能替代人工逐个审核。

功能词 the、and、i、to、of、is、it、that 的速率熵都在 0.95 以上（the 0.9748、and 0.9789），内容词 love、king、heart、sweet 落在 0.82–0.93（0.9271、0.8260、0.8313、0.8925），只出现在一篇文档的专名 osric、banquo 按定义为 0。0.93 的线正好切在功能词与内容词之间；`man` 的速率熵 0.9388 略高于阈值线，但文档频率比值只有 0.2359，被” 宽” 这一条件挡在候选带外——三个条件是” 与” 关系，速率熵补上的正是前两个特征缺失的” 分布是否均匀” 这一维。

最终停用词的选择是 `the and i to of a my that in is not me it for with be his this but have as so what by no all do if are on from or which more then their when than yet`。

2.2 倒排索引与查询结果

2.2.1 倒排索引构建结果

任务 2 关注索引是否反映停用词过滤与词干化后的词项集合，以及这些处理对索引规模的影响。表 6 给出四种配置的规模与构建代价。

表 6: 四种构建配置的索引规模与构建代价（主语料，195 篇）

指标	原始配置	仅停用词	仅词干化	正式（停用词＋词干化）
处理标志	`(raw)`	`stopwords`	`stemmed`	`stopwords stemmed`
词元数	866 626	556 866	866 626	556 866
词项数	25 705	25 666	14 947	14 937
倒排记录数	152 650	147 909	121 903	117 704
位置数	866 626	556 866	866 626	556 866
索引字节	5 598 793	4 320 622	4 958 598	**3 685 655**
内建构建时间	0.20 s	0.18 s	0.23 s	0.19 s
相对原始配置	100%	77.17%	88.57%	**65.83%**

注：处理标志由索引头写出并由 `dump`/`query` 头部打印；倒排记录数为词项与文档的出现对数，位置数为位置总数（等于保留词元数）；内建构建时间为 C++ 内部计时（正式配置取 3 次中位）；相对原始配置为索引字节之比。

停用词过滤只删词元、几乎不动词表(词项仅少 39 个),体积却降到原始配置的 77.17%:省的是倒排记录区与位置数组,它们的大小正比于保留词元数。词干化不删词元、位置仍为 866 626,但词表压到 14 947 个,体积降到 88.57%,省的是词项表与倒排记录条数(152 650 降到 121 903)。两者叠加的正式配置是原始配置的 65.83%(3 685 655 字节)。位置总数等于词元总数、每条倒排记录项的词频等于其位置数,这两条恒等式在四种配置上都成立,也解释了为什么位置数组会随停用词过滤明显缩小。

构建可重复:正式配置连续构建 3 次,外部墙钟 0.2563/0.2908/0.2583 s、内建计时 0.19/0.22/0.19 s,中位为 0.19 s。构建时间有小幅波动,但三次都处于同一量级。

表 7 用同一套插桩计数比较三种词典实现。

表 7: 三种词典实现的构建代价(正式配置,各构建 3 次)

词典	比较次数	旋转次数	再哈希次数	构建时间(内建 / 外部中位)
伸展树(主)	6 392 724	5 850 795	0	0.19 s / 0.2610 s
哈希表	688 494	0	11	0.09 s / **0.1603 s**
`std::map`(对照)	9 435 694	0	0	0.14 s / 0.2084 s

注:比较次数、旋转次数、再哈希次数为 C++ 内建计数;外部墙钟含进程启动与文件读写,受并行负载影响只作量级比较。

三种词典的差别只在构建过程:哈希比较次数约为伸展树的 10.8%、构建最快,伸展树有 5 850 795 次旋转,`std::map` 比较次数最多。哈希适合追求构建速度,伸展树更适合需要有序输出的场景;本项目的结构取舍见第 4.2 节。第二版索引格式把正式索引从 3 685 655 字节降到 1 725 694 字节(46.82%,省 53.18%),原因见第 3.4 节。

2.2.2 查询结果

任务 3 的验收点是" 给定单词或短语,返回包含它的文档 ID",并要求结果可解释。本节用正式索引(`stopwords stemmed`)给出代表查询,原始索引作对照,表 8 是汇总。

这张表覆盖查询侧的全部语义分支。短语查询验证 "相邻而非共现"(`king richard` 命中 7 篇,前 3 篇均命中 2/2 个词);布尔查询展示 `--min-match` 的松紧(`king queen` 默认要求全部命中,返回 33 篇;`--min-match 1` 返回 43 篇)。两类空结果被明确区分:`the` 为空是停用词未建索引,未登录词为空是词典里没有,说明信息会给出原因。原始索引的对照说明解释器完全由索引头处理标志驱动:同一个 `--word the` 在原始索引上返回 3 篇;`loves` 在正式索引上归一为 `love`、返回 138 篇,在原始索引上按原形只返回 43 篇。

程序还提供结构化输出。JSON 顶层字段为查询、模式、词项、命中(每项包含文档、命中词数和词频)与说明(被丢弃词项及原因),代码中对应:

```
{"query":"love","mode":"word","terms":["love"],
 "hits":[{"doc":16,"matched":1,"tf":195}, ...],"notes":[]}
```

短语查询只把 `mode` 换成 `phrase`;含停用词的短语在 `notes` 中记录原因,这是阈值实验能逐条归因的基础。

单次查询的墙钟中位数为 19.2 ms(N=201,最小 17.1 ms、最大 29.9 ms,均值 19.60 ms),口径是包含进程启动与索引加载的端到端时间;201 次查询合计 4.05 s。绝对时间受并行负载影

表 8: 代表性查询的实测输出（正式索引过滤停用词并做词干化，原始索引两者均不做）

查询	索引 / 参数	归一后词项	结果数	说明与代表输出
`--word osric`	正式，`--pretty`	`osric`	1	文档 30《Hamlet》；命中行 `> Osric, who brings back to him that you attend him in`
`--word love`	正式	`love`	138	词频最高：文档 16（195）、文档 35（165）、文档 2（142）
`--phrase "king richard"`	正式，`--top-x 3`	`king, richard`	3	基线 7 篇；前 3 篇命中 2/2 个词，词频总和 193/191/140
`--phrase "my lord"`	正式	`lord`	42	`my` 是停用词，打印退化说明后只查 `lord`
`--words "king queen"`	正式，`--min-match 2`	`king, queen`	5	截断前 33 篇（全部命中）；`--min-match 1` 为 43 篇
`--word the`	正式	（无）	0	说　明：`the -- stop word (excluded from the index)`
`--word notaword`	正式	（无）	0	说　明：`notaword -- not in the index`
`--word loves`	正式 / 原始	`love` / `loves`	138 / 43	词干化把 `loves` 归一到 `love`；原始索引上 `love` 为 127 篇
`--word the`	原始	`the`	3	原始索引未过滤停用词，同一查询可检索

注：结果数分母为 195 篇；单词与布尔查询按命中词数降序、词频总和降序、文档编号升序排序；`--pretty` 的命中行来自文本层行号映射。

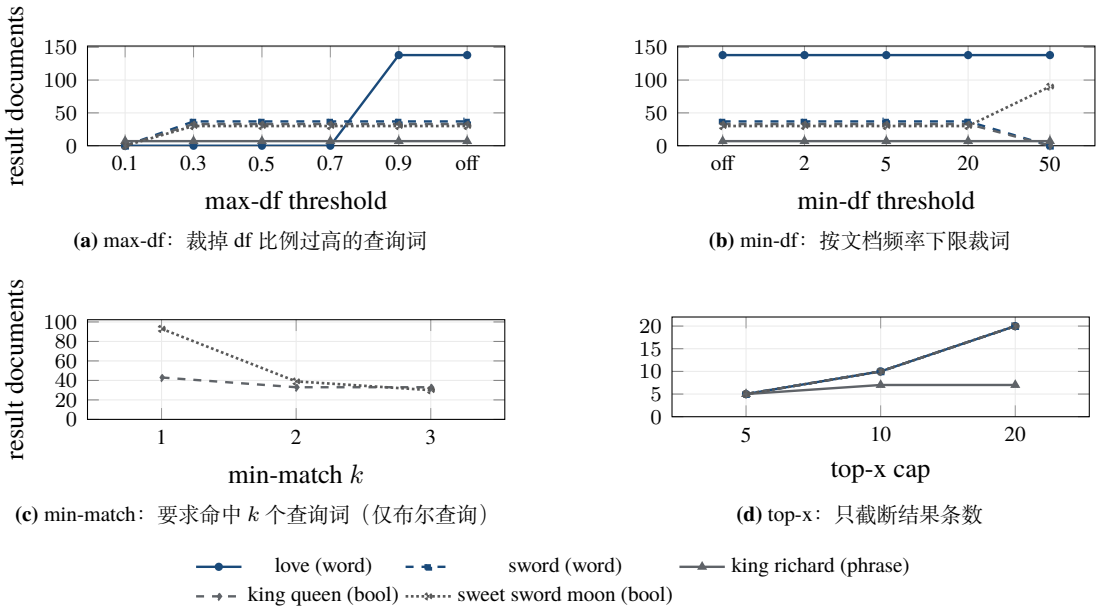
响有约 ±20% 波动，只宜作量级参考；查询结果是确定性的。

2.3 阈值实验与复杂度分析

2.3.1 阈值实验结果

四个阈值参数全部作用在查询侧、默认关闭，”不带参数的查询” 就是基线：`--max-df` 丢弃文档频率比值过高的查询词，`--min-df` 丢弃文档频率过低的词，`--min-match` 控制多词查询至少命中几个词，`--top-x` 在排序后截断结果（语义与默认值见表 1）。本节回答它们分别把结果集改变了多少、改变方式是否符合设计语义。

实验采用 “一次只动一个参数”：15 条查询（高频词 love/sweet、中频词 good/night、低频词 sword/moon、专名 banquo/osric、短语 king richard、含停用词短语 my lord、布尔查询 king queen 与 sweet sword moon、停用词 the/of），每条取无阈值结果为基线，再分别在最大文档频率（5 档）、最小文档频率（4 档）、结果截断（3 档）、最少命中词数（3 档）下重跑，共 201 条测量（15 条基线，加前三类共 180 条、最少命中词数 6 条），每条记录结果数、与基线的 Jaccard 相似度、被丢弃词数（说明条目数）与耗时。全部测量共用同一份正式索引，CSV 为 202 行；图 5 给出四个参数对返回文档数的影响。



(a) max-df：裁掉 df 比例过高的查询词

(b) min-df：按文档频率下限裁词

(c) min-match：要求命中 k 个查询词（仅布尔查询）

(d) top-x：只截断结果条数

图 5: 四个阈值改变的是查询语义而非结果数量：max-df/min-df 裁掉查询词（单词查询结果可在临界档归零，双词语句裁词后结果反而变多），min-match 改布尔条件，top-x 只截断条数；短语查询对前两个参数不敏感。

表 9 抽出代表性读数，覆盖” 参数收紧使结果变少、变多、不变” 三种情况。

最重要的读数是” 反直觉” 的那一行：`--min-df 50` 作用在 sweet sword moon 上，结果从 30 篇增加到 90 篇。原因不是精度提高，而是语义变了：sword 与 moon 的文档频率分别为 37、35，低于阈值被丢，剩下唯一的 sweet 使三词全部命中退化成单词查询，而 sweet 单独就有 90 篇。同类现象在最大文档频率参数上表现为 “全丢”：love 在 0.7 档、sweet 在 0.3 档、单词查询被自身文档频率阈值丢弃后返回 0。最少命中词数按设计在全部命中与任意命中之间移动，结果截断则是纯截断，Jaccard 近似等于 x 与基线结果数之比。

表 9: 阈值矩阵的代表性读数：基线返回数、取值后返回数与 Jaccard 相似度

查询	参数与取值	基线 → 结果	Jaccard	解释
love（单词查询）	max-df 0.9 / 0.7	138 → 138 / 0	1.00 / 0.00	文档频率比值 0.7077，0.7 档起被丢（说明 1 条）
sweet（单词查询）	max-df 0.3 / 0.1	90 → 90 / 0	1.00 / 0.00	文档频率比值 0.4615，0.3 档起被丢
good / night	max-df 0.1	57 / 56 → 0	0.00	文档频率比值约 0.29，仅最严档丢弃
king queen（布尔查询）	max-df 0.1	33 → 0	0.00	任一词被丢则全部命中条件失败
sweet sword moon	min-df 50	30 → 90	0.3333	sword/moon（文档频率 37/35）被丢，全部命中条件退化为单词 sweet（说明 2 条）
sword / moon	min-df 50	37 / 35 → 0	0.00	单词查询被自身文档频率阈值丢弃
banquo / osric	min-df 2	1 → 0	0.00	文档频率为 1，小于 2
king queen（布尔查询）	min-match 1/2/3	33 → 43/33/33	0.7674/1.00/1.00	1 为并集；k 超过词项数时按 2 计
sweet sword moon	min-match 1/2/3	30 → 93/39/30	0.3226/0.7692/1.00	1 为并集，2 为任两词，3 为全部命中
love（单词查询）	top-x 5/10/20	138 → 5/10/20	0.0362/0.0725/0.1449	纯截断，Jaccard 近似 $x/138$
king richard（短语查询）	max-df/min-df 全部档	7 → 7	1.00	短语绕过词项筛选，语义要求每词都在
the / of（停用词）	全部档	0 → 0	1.00	停用词不可检索，与阈值无关（说明不变）

注：Jaccard 为结果文档集合与基线集合的交并比；” 基线” 是同一查询在无阈值时的返回数。

两处“不受影响”同样重要：短语查询的 Jaccard 在所有文档频率阈值档上恒为 1.00，因为短语必须整段匹配、丢弃任何词都会改变含义（要么全有、要么失败）；停用词查询 `the`/`of` 恒为空并给出说明，也与阈值无关。201 条测量中 Jaccard 为 0 的有 23 条，分别来自最大文档频率阈值过严时的 love、sweet、good、night、sword、moon 与相关布尔查询，或最小文档频率阈值过严时的 banquo、osric 等词；全部是“查询词被自身阈值丢弃”。说明条目数与丢弃词数一致：最大文档频率 0.9、0.7、0.5 档各有 3 到 4 条说明，0.3 档 6 条，0.1 档中 9 条为一个词、1 条为两个词、1 条为三个词；最小文档频率 2、5、20 档各有 5 条说明，50 档有 7 条一个词、2 条两个词。关闭阈值与结果截断档的说明来自 my lord、the、of 三个含停用词的查询。每条变化都能归因到具体被丢弃的词项。

结论：阈值改变的是查询语义而不是单纯精度：丢弃查询词会改变约束（全部命中退化、单词查询归零），最少命中词数在全部命中与任意命中之间移动，结果截断只减少返回数量；短语查询与停用词查询在词项筛选下保持不变。展开与对” 检索质量” 的含义见第 4.2 节。

2.3.2 复杂度分析

主要操作的时间与空间复杂度见表 10。表中的 N、T、P、D、k、R、B 分别表示词元数、词项数、倒排记录数、文档数、查询词数、临时块数与块预算。统计成本以一次词元扫描为主；查询成本由词典定位、倒排记录遍历和最终排序组成；磁盘词典用页读次数替代常驻内存。

表 10: 主要操作的时间与空间复杂度

操作	时间	空间	说明
建索引（内存）	$O(N + \sum \mathrm{df} \cdot C)$	$O(T + P + N)$	C 为词典比较成本
建索引（块式）	$O(N + RB \log B + TR)$	$O(B +$ 块内倒排记录$)$	B 为活词预算
单词查询	$O(\log T + \mathrm{df})$	$O(D)$	二分定位后遍历倒排记录
短语查询	$O(\mathrm{df}_{\min} \cdot \mathrm{tf}_1 \cdot k \log \mathrm{tf})$	$O(D + k)$	稀有词候选加位置验证
布尔查询	$O(\sum \mathrm{df} + D \log D)$	$O(D)$	累加、过滤与排序
第二版索引编码	$O(\sum \mathrm{df} + \sum$ 位置$)$	$O($单块$)$	与第一版同阶，常数更大
B+ 树查找	$O(h \log F)$	$O($缓存页$)$	h 为树高，F 为扇出

这些复杂度解释了后续实验结果中的几个现象：建索引的主要成本是词元扫描和词典访问；短语查询先选择文档频率最小的词，能显著减少候选文档；块式构建限制的是活词数量，但临时块数增加会把成本转移到归并；磁盘 B+ 树用少量页读换取词典不必常驻内存。

3 Bonus 的结果与分析

附加题有两条相互独立的规模轴：500,000 个文件主要压测构建侧和文档表，400,000,000 个不同词主要压测词典与倒排记录。项目分别实现了 SPIMI 式块式构建、磁盘 B+ 树词典和第二版索引格式的差分压缩，并完成了能够实际运行的规模实验。

3.1 规模边界与实测范围

按本轮 500,000 文档合成语料的实测锚点，4,908,747 个词项的整份内存索引峰值内存为 2.00 GiB。若只按词项数线性放大到 4 亿词，峰值将达到约 160 GiB，普通单机无法容纳。因此

全内存构建不能直接回答附加题。

500,000 文件这一轴已经实测完成：统计、块式构建和内存构建都在 16 GiB 机器上运行，没有出现文件数导致的地址空间或文档表问题。400,000,000 个不同词没有端到端跑完，查询侧用 B+ 树实测到 50,000,000 个键后外推，构建侧用块式构建的 490 万词实测和 O(预算) 内存上界说明扩展路径。报告会明确区分实测点与外推点。

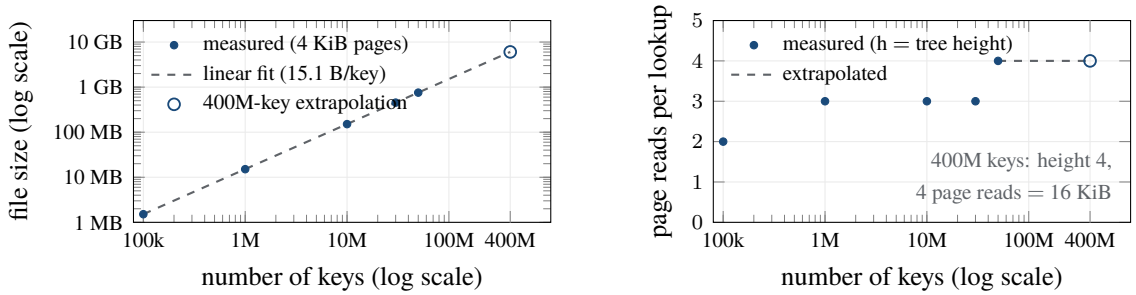
3.2 磁盘 B+ 树词典

查询侧的问题不是如何查得更快，而是如何避免把完整词项表读入内存。B+ 树附属词典文件把每个词项映射到倒排记录偏移和块长，查询一次只经过根到叶的若干页。本轮从 10 万键实测到 5,000 万键，结果见表 11。

表 11: B+ 树词典的规模实测（4 KiB 页，批量装载，1,000 次随机查询全部校验）

键数	树高	总页数	文件字节	字节/键	每查询页读
100 000	2	369	1 515 520	15.1552	2.00
1 000 000	3	3 688	15 110 144	15.1101	3.00
10 000 000	3	36 865	151 003 136	15.1003	3.00
30 000 000	3	110 593	452 993 024	15.0998	3.00
50 000 000	4	184 322	754 987 008	15.0997	4.00

注：页读为缓存 1 页时的实际读页数，等于树高。



(a) 文件字节 = 15.1 B/键 × 键数，严格线性

(b) 每次查询只读树高页：4 亿键也只需 4 页 = 16 KiB

图 6: B+ 树词典的规模实验：文件体积严格线性（15.1 B/键），而每次随机查询的页读等于树高、随高度对数增长；按实测扇外出推，4 亿个不同词只需 4 层、每次查询 4 页（16 KiB），文件约 6.04 GB。

五个实测点的字节/键保持在 15.10–15.16 之间，体积近似严格线性。扇出约 330–370，平均每叶约 272 个键，因此 4 层已经覆盖 10^8 级键。按实测趋势外推 4 亿键，文件约 6.04 GB，树高仍为 4，每次查询读 4 页，即 16 KiB。树高与页读在 5,000 万键时已经实测到 4 层，外推的只有文件字节和页数。

3.3 块式构建与 500,000 文档

正式莎士比亚语料上，预算 4,096 的块式构建产生 17 个临时块、峰值活词 5,129，临时块共 4,166,556 字节，产物与内存构建逐字节一致。

500,000 文档实验把这条路径放大到实际小文件规模。合成语料包含 500,000 个文件、20,000,000 个词元和 4,908,747 个不同词。块式构建产生 4,855 个临时块，扫描 5.47 秒、归并

125.08 秒，总墙钟 168.16 秒，峰值内存 1.01 GiB；全内存构建耗时 76.75 秒、峰值内存 2.00 GiB。两种方式生成的第一版索引都是 427,603,247 字节，SHA-256 相同。

表 12: 500,000 文档 / 2,000 万词元的构建结果

构建方式	索引字节	构建阶段	峰值内存	产物
块式第一版	427 603 247	扫描 5.47 + 归并 125.08 s	1.01 GiB	SHA-256 相同
块式第二版	284 560 009	扫描 5.42 + 归并 123.31 s	1.46 GiB	差分压缩
内存第一版	427 603 247	内建 44.93 s，总 76.75 s	2.00 GiB	基准

注：块式与内存第一版索引的 `cmp` 和 SHA-256 均一致。

块式构建把峰值内存几乎减少一半，但归并耗时明显增加。4,855 个临时块当前通过线性扫描所有游标找最小词，成本约为 $O(T \cdot R)$；换成堆做 k 路归并可以降到 $O(T \log R)$。这部分时间与临时空间是大规模构建必须付出的工程代价。

3.4 第二版索引格式的压缩结果

第二版索引格式只改变倒排记录区：文档编号和位置存差分，词频与差分值使用字节对齐变长码。主语料索引从 3,685,655 字节降到 1,725,694 字节，为第一版格式的 46.82%；构建耗时仍为 0.19 秒，查询输出不变。

500,000 文档语料上，倒排记录区在总文件中占比只有 56.1%，词项表成为主要体积来源，所以整体压缩率不如主语料：第一版 427,603,247 字节降到第二版 284,560,009 字节，为第一版的 66.55%。倒排记录区自身的压缩率约为 59.6%，与主语料的 61.8% 接近。

表 13: 第二版索引格式的压缩结果与主要成本

语料	第一版字节	第二版字节	第二版/第一版	节省	压缩构建峰值内存
莎士比亚	3 685 655	1 725 694	46.82%	53.18%	0.19 s（内建）
500k 合成	427 603 247	284 560 009	66.55%	33.45%	1.46 GiB（第一版为 1.01）

注：500k 语料的压缩率受词项表占比影响；第二版格式不压缩词项表。

压缩适合存储受限、CPU 与构建内存相对充裕的场景。主语的 3.7 MB 索引可以整体缓存，第二版索引的收益主要在磁盘和传输；500k 语料压缩构建的峰值内存从 1.01 GiB 升到 1.46 GiB，说明压缩并非在所有资源维度上都占优。

3.5 Bonus 结论

500,000 个文件这一轴已经实测通过；400,000,000 个不同词没有端到端运行。查询侧已经把词典移到磁盘并把单次查询控制在常数页读，构建侧已经用块式构建去掉“内存随全词表增长”的假设。完整的大规模系统还缺 SPIMI 归并与 B+ 树的直接集成、分布式分片与故障恢复。现有结果支持“关键部件可扩展”的结论，但不支持“完整 4 亿词系统已经跑通”的说法。

4 项目评价

前面的结果证明了当前实现能完成课程任务，也给出了规模扩展的两条路径。项目评价主要回答两个问题：同一套引擎能否换一份语料工作，以及当前实现距离更通用的搜索引擎还差什么。

4.1 推广：第二语料 RFC

项目另备第二语料：IETF RFC 1035（DNS）、5321（SMTP）、7231（HTTP 语义）。三篇文档通过通用导入器转成纯文本格式，语料清单与文本层是唯一接口；同一套 C++ 二进制没有修改任何代码，即可完成统计、建索引、查询和独立对拍。

表 14: 第二语料 RFC 的端到端实测

指标	原始配置（无过滤、无词干）	正式配置（58 词停用词 + 词干化）
文档数	3	3
词元数	75 250	45 370
词项数	4 328	2 660
倒排记录数	7 006	4 594
索引字节	510 181	308 164
`brute_force.py` 对拍	4 328 词全项一致	2 660 词全项一致

注：RFC 语料共 9 946 行、81 848 个空白切分词。

RFC 语料最能说明停用词没有万能表。用与莎士比亚相同的三特征规则 $(0.40, 0.001, 0.93)$，RFC 本轮推导出 58 个停用词，与莎士比亚 39 词表只重叠 26 个。RFC 2119 的 `must`、`may`、`should` 覆盖三篇文档，df 比值都是 1，但速率熵只有 0.809、0.899、0.885，低于 0.93，因此没有被误判为停用词；`server`、`page`、`rfc` 的速率熵接近 1，反而被收入推导表。

查询侧也按预期工作：`zone` 命中 3 篇，`postmaster` 命中 SMTP 一篇并归一为 `postmast`；`name server` 中 `server` 被 58 词表过滤，程序用说明信息解释后只查 `name`。这说明概念上“引擎部分”和“制作语料部分”已经分开，换语料需要新的 producer 规则和停用词阈值，但不需要改索引与查询代码。

推广也有边界。RFC 的纯文本导入没有做分页脚注清洗，`[Page N]` 中的 `page` 进入词表并成为停用词；这个噪声属于 producer 侧，而不是引擎侧。换语料时仍要重新检查分词、文档粒度和停用词分布，不能认为“同一二进制可运行”等于“无需任何数据工程”。

4.2 局限与改进

索引与查询部分的主要边界见表 15。其中短语语义和排序模型影响最大：位置定义在过滤后的词元流上，因此 `my lord` 会退化为只查 `lord`；全由停用词组成的短语返回空。排序只使用命中词数和 tf 总和，没有 idf 或长度归一化，长剧更容易占据靠前位置。

表 15: 系统局限、影响与改进方向

局限	影响	改进方向
短语在过滤后词元流上匹配	含停用词的短语退化；全停用词短语返回空	保留停用词位置，或为短语另建有停用词的索引
切词器仅支持 ASCII	Unicode 语料不能直接使用	UTF-8 解码与 Unicode 类别切词，并重跑全部对拍
纯文本导入无清洗	RFC 页码等样板文字进入词表	清洗放在 producer 侧，保持可审计、可关闭
排序只用命中词数与词频总和	没有逆文档频率与长度归一化	实现 TF-IDF/BM25，并重新设计相关性实验
查询与构建为单线程	201 次阈值实验串行约 4.05 s	倒排记录并行扫描；构建端按块并行
文档粒度固定为整篇作品	scene 级粒度没有正式接入选型实验	粒度参数化，比较 scene 与合并短诗的停用词分布
无增量索引	更新一篇文档需要重建整份索引	段式索引与定期合并
单机、无分布式	4 亿词的真实部署仍受构建时间和磁盘限制	按词范围分片构建与查询

词典选择的结论也应当如实表述。哈希在纯构建速度上明显快于伸展树；正式配置下伸展树约 0.19 s、哈希约 0.09 s，但伸展树中序遍历直接得到有序词项表，并且符合课程对自实现数据结构的要求。三种实现写出的索引逐字节相同，说明词典结构改变的是构建过程，不改变持久化结果。

阈值部分最容易被误解的是“参数收紧会使结果减少”。实际上 `--max-df` 与 `--min-df` 先筛选查询词，可能让单词查询归零，也可能使多词全部命中条件退化后结果变多。`--min-match` 改变布尔条件，`--top-x` 才是结果截断。短语查询不受词侧阈值影响，这是短语必须整段匹配的语义结果。

从项目本身看，最有价值的不是某一个墙钟数字，而是可复现的验证链：同一份文本层能由独立 Python 实现重建倒排记录，三种词典能写出完全相同字节，块式构建与内存构建一致，第二版索引格式能由独立解析器还原，磁盘词典与全量加载逐词一致。这些约束把很多“最后看起来能跑”的实现错误提前暴露出来。

后续工作按收益排序：先补 scene 级和合并十四行诗的粒度实验，再补 BM25/TF-IDF 与相关性评价，然后把 SPIMI 的归并输出直接写成 B+ 树，跑一次真正的 500k/400M 端到端构建。分布式分片、增量索引和 Unicode 切词属于下一阶段，不应与当前已经验证的单机范围混在一起表述。

5 参考文献

[1] MIT Shakespeare. *The Complete Works of William Shakespeare*. http://shakespeare.mit.edu/；官方 GitHub 镜像 https://github.com/TheMITTech/shakespeare.

[2] M. F. Porter. An algorithm for suffix stripping. *Program*, 14(3):130–137, 1980.

[3] M. F. Porter. *The Porter Stemming Algorithm: canonical C implementation*. https://tartarus.org/martin/PorterStemmer/c.txt.

[4] NLTK Project. *nltk.stem.PorterStemmer*. https://github.com/nltk/nltk/blob/3.9.1/nltk/stem/porter.py.

[5] Snowball Project. *The English (Porter2) stemming algorithm*. https://snowballstem.org/algorithms/english/stemmer.html.